



Ministry of Communications and Information Technology (MCIT)

Digital Egypt Pioneers Initiative (DEPI) — Round 4

Frontend Development Track — React.js

Instructor: Dr / Nasser Khairy

Software Requirements Specifications (SRS)

CareConnect — Online Healthcare Booking Platform

Frontend React.js Development Project

Field	Detail
Project Name	CareConnect
Repository	https://github.com/TomasNageh/CareConnect
Document Version	2.0
Date	July 2026

Team Members

#	Full Name	Role
1	Tomas Nageh	Team Lead / Frontend Developer — Authentication & Patient Experience
2	Omar Abdelaziz	Frontend Developer — Admin Console & System Integration
3	Jolie Fayed	Frontend Developer — Doctor Experience & Messaging
4	Jana Khaled	Frontend Developer / QA — Public Discovery & Search

Table of Contents

Team Members	1
Table of Contents	2
Revision History	4
1. Project Planning & Management	5
1.1 Project Overview	5
1.2 Scope & Authorship Statement	5
1.3 Roles & Responsibilities	5
1.4 Risk Assessment & Mitigation Plan	6
1.5 Success Criteria	6
2. Literature Review & Technology Stack	7
2.1 Background & Domain Analysis	7
2.2 Technology Stack	7
2.3 Comparative Analysis of Similar Platforms	7
2.4 Self-Evaluation Criteria	8
3. Requirements Gathering	9
3.1 Stakeholder Analysis	9
3.2 User Stories	9
3.3 Functional Requirements	10
A. Authentication & Account Requirements	10
B. Patient Requirements	10
C. Doctor Requirements	11
D. Admin Requirements	11
E. Messaging Requirements	12
3.4 Non-Functional Requirements	12
4. System Analysis & Design	14
4.1 Problem Statement & Objectives	14
4.2 Use Case Descriptions	15
UC-01 — Book Appointment	15
UC-02 — Search & View Doctor Profile	15
UC-03 — Doctor Verification	15
UC-04 — Doctor–Patient Messaging	16
4.3 Software Architecture	16
4.4 Route → Component Map	16
4.5 Data Model	17
4.6 API Reference	18
Auth	18

Patient	18
Doctor	18
Admin	19
Chat	19
5. Implementation	20
5.1 Repository Structure	20
5.2 Coding Notes	20
5.3 Environment & Execution	20
5.4 Demo Accounts	21
6. Testing & Quality Assurance	22
6.1 Manual Test Plan	22
6.2 Known Limitations	22
7. Final Presentation & Reports	23
7.1 User Manual (Patient Journey)	23
7.2 User Manual (Doctor & Admin Journeys)	23
7.3 Future Enhancements	23
7.4 Technical Summary	23
Appendix A: Glossary	24
Appendix B: Demo Accounts (Quick Reference)	24
Appendix C: References	24

Revision History

Version	Date	Description	Author
2.0	July 2026	Initial SRS: project planning, literature review, requirements, system design, implementation, testing, and final reporting sections.	Development Team

1. Project Planning & Management

1.1 Project Overview

CareConnect is a healthcare appointment booking platform composed of a React 18 single-page frontend and a Node.js/Express mock REST API. It allows patients to search for doctors and clinics, book and manage appointments, message their doctors, and maintain a personal medical history; it allows doctors to manage clinics, availability slots, and appointments; and it allows administrators to verify doctors and clinics, manage user accounts, and resolve patient complaints.

The project was built by a four-person team to practice full-stack development on a realistic multi-role domain: core React patterns (hooks, context, routing, forms) on the frontend, and a small Express/JWT REST API on the backend. Parts of the backend and auth wiring were AI-assisted to unblock frontend development, since server-side authentication and API design were not the primary focus of the team's coursework.

1.2 Scope & Authorship Statement

In the interest of an accurate SRS, the following table records what was hand-written versus AI-assisted or template-derived, matching the project README:

Part	Origin
Pages, forms, routing, useState/useEffect logic	Written by the team (beginner-to-intermediate React)
API service layer (services/*.js, Axios calls)	Written by the team with basic Axios usage
components/ui (shadcn/Radix primitives)	Copied from the shadcn/ui template — not written from scratch
Express backend and JWT authentication	AI-assisted — outside the scope of the team's React coursework
AuthContext and Axios interceptors	AI-assisted

This transparency matters for grading and for future maintenance: the parts most likely to need review or hardening before any real-world use are the AI-assisted backend and auth layers, since they have not been production-audited.

1.3 Roles & Responsibilities

CareConnect was built by a four-person team, with ownership split by feature area of the codebase as recorded in the project's team division plan.

Track	Responsibilities	Owner
Authentication & patient experience	Login/registration, AuthContext (JWT session state), booking flow, patient dashboard (appointments, history, reviews, notifications)	Tomas Nageh
Admin console & integration	App routing & route guards, Axios/API layer (JWT interceptor), admin dashboard (users, verifications, analytics), Express mock backend integration (AI-assisted)	Omar Abdelaziz
Doctor experience & messaging	Doctor dashboard, clinic and slot management, REST-based chat (conversations & messaging)	Jolie Faye

Track	Responsibilities	Owner
Public discovery & QA	Homepage composition, doctor/clinic search & profile pages, manual test plan	Jana Khaled

1.4 Risk Assessment & Mitigation Plan

#	Risk	Probability	Impact	Mitigation Strategy
1	In-memory data store loses all state on server restart	High	Medium	Documented clearly in README as a known limitation; acceptable for a learning/demo project, but flagged as a blocker before any real deployment
2	AI-assisted backend/auth code contains unreviewed security gaps (e.g. plaintext password comparison)	Medium	High	Explicitly called out in Section 1.2; recommended follow-up: bcrypt password hashing, refresh tokens, and rate limiting before any production use
3	Single-developer bandwidth limits scope	Medium	Medium	Feature set intentionally scoped to core booking/chat/admin flows; no payments, video calls, or notifications integrations
4	shadcn/Radix template components mask gaps in the developer's own component-authoring practice	Low	Low	Template usage disclosed; developer wrote all page-level composition around the primitives
5	No automated test suite currently exists	High	Medium	Manual test plan defined in Section 6; automated tests listed as a future enhancement in Section 7.3

1.5 Success Criteria

#	Criterion	Target	Measurement Method
1	Core booking flow works end-to-end	Patient can search → view doctor → pick slot → confirm booking without errors	Manual walkthrough with demo accounts
2	Role-based access is enforced	Patient/Doctor/Admin routes reject the wrong role and redirect appropriately	Manual test of ProtectedRoute for each role
3	Doctor/clinic verification workflow functions	Admin can approve a pending doctor or clinic and it becomes bookable	Manual test using dr_pending demo account
4	Responsive layout	No horizontal scroll or broken layout at mobile/tablet/desktop widths	Browser devtools responsive check
5	Zero ESLint errors	npm run lint passes clean	CI or local lint run before each commit

2. Literature Review & Technology Stack

2.1 Background & Domain Analysis

Online medical appointment booking has matured from static clinic listings into interactive platforms offering doctor discovery, real-time slot availability, and in-app communication between patients and providers. Multi-role systems (patient / provider / administrator) are now the norm because they let platform operators verify practitioner credentials before exposing them to patients — a pattern CareConnect follows directly with its doctor and clinic verification workflow.

CareConnect narrows this domain to a learning-appropriate but realistic slice: authentication, doctor/clinic discovery, slot-based booking, reviews, medical history, notifications, direct messaging, and an admin console — without payment processing, insurance integration, or telemedicine video, which are common in commercial platforms but out of scope here.

2.2 Technology Stack

Layer	Technology	Version	Purpose
Frontend framework	React	18.3.x	Component-based SPA UI
Build tool	Vite	6.3.x	Dev server + production bundling
Routing	React Router DOM	7.11.x	Client-side routing, protected routes
Styling	Tailwind CSS	4.1.x	Utility-first styling
UI primitives	Radix UI (via shadcn/ui)	1.x components	Accessible unstyled primitives: dialog, dropdown, tabs, select, avatar, etc.
HTTP client	Axios	1.7.x	REST calls to the mock API
Toasts	sonner	2.0.x	Non-blocking success/error notifications
Date picker	react-day-picker	8.10.x	Slot/date selection UI
Icons	lucide-react	0.487.x	Icon set
Backend runtime	Node.js + Express	Express 4.21.x	Mock REST API server
Auth	jsonwebtoken (JWT)	9.0.x	Stateless bearer-token authentication
Cross-origin	cors	2.8.x	CORS handling between :5173 and :3001
Data persistence	In-memory JS arrays	—	No database — data resets on server restart

2.3 Comparative Analysis of Similar Platforms

Platform	Key Features	Limitations	Lessons Applied to CareConnect
Vezeeta (Egypt/MENA)	Doctor search, booking, ratings, insurance filter	Feature-rich UX is complex for a learning clone to replicate fully	Kept doctor cards, specialty search, and star ratings; simplified booking to a slot picker without insurance logic
Zocdoc (USA)	Real-time slot availability, verified reviews, in-network	Requires production-grade backend and	Adopted the slot-availability model (per-clinic time slots) and

Platform	Key Features	Limitations	Lessons Applied to CareConnect
	filtering	payment/insurance integration	verified-doctor badge concept
Practo (India)	Multi-clinic doctors, patient records, teleconsultation	Teleconsultation and EHR depth beyond project scope	Modeled the one-doctor-to-many-clinics relationship and a lightweight medical history record

2.4 Self-Evaluation Criteria

As a self-directed project (not tied to a formal grading rubric), CareConnect is evaluated against the following criteria, adapted from common frontend bootcamp/course rubrics:

- Component architecture: reasonable decomposition into pages, feature-scoped components, and shared UI primitives.
- State management: correct use of `useState/useEffect` and a single `AuthContext` for session state — no premature adoption of `Redux`.
- Routing: role-aware protected routes with sensible redirects for unauthenticated or wrong-role access.
- Functionality completeness: booking, search, chat, dashboards, and admin verification all implemented end-to-end against the mock API.
- Code honesty: clear documentation of which parts are hand-written vs. templated vs. AI-assisted.

3. Requirements Gathering

3.1 Stakeholder Analysis

Stakeholder	Role	Interest in System	Priority
Patient	Primary end user	Finds doctors/clinics, books appointments, messages doctors, tracks medical history	High
Doctor	Provider / secondary user	Manages clinics, working hours, slots, appointments, and patient messages	High
Admin	Platform operator	Verifies doctors/clinics, manages user accounts, resolves complaints, views analytics	High
Clinic (entity)	Data entity, not a login role	Represents a physical location tied to one doctor with its own operating hours and slots	Medium

3.2 User Stories

Stories are written as: As a [role], I want to [goal], so that [benefit]. Each maps to the page or API route that implements it.

ID	User Story	Priority	Implementation
US-01	As a visitor, I want to search doctors by specialty, location, and rating so I can find the right provider.	Must Have	DoctorSearchPage → GET /api/patient/doctors/search
US-02	As a visitor, I want to browse clinics so I can find a provider near me.	Must Have	ClinicSearchPage / ClinicPage
US-03	As a visitor, I want to register as a patient or a doctor so I can use the platform.	Must Have	RegisterPage → POST /api/auth/register/patient doctor
US-04	As a user, I want to log in with email and password so I can access my dashboard.	Must Have	LoginPage → POST /api/auth/login
US-05	As a patient, I want to view a doctor's profile, clinics, and available slots so I can choose when to book.	Must Have	DoctorProfilePage → GET /api/patient/doctors/:id/slots
US-06	As a patient, I want to complete a multi-step booking flow (clinic → slot → confirm) so I can reserve an appointment.	Must Have	BookingFlow → POST /api/patient/appointments
US-07	As a patient, I want to view, update, or cancel my appointments so I can manage my schedule.	Must Have	PatientDashboard → GET/PUT/DELETE /api/patient/appointments
US-08	As a patient, I want to leave a review for a doctor so other patients can benefit from my experience.	Should Have	PatientDashboard → POST /api/patient/reviews
US-09	As a patient, I want to keep a personal medical history log so I can track past diagnoses and treatment.	Should Have	PatientDashboard → /api/patient/medical-history
US-10	As a patient or doctor, I want to message the other party so we can communicate outside appointments.	Must Have	ChatPage → /api/chat/*
US-11	As a patient or doctor, I want to see notifications about appointment activity so I stay informed.	Should Have	/api/patient/notifications, /api/doctor/notifications
US-12	As a doctor, I want to register and manage one or more	Must Have	DoctorDashboard →

ID	User Story	Priority	Implementation
	clinics so patients can find and book me there.		/api/doctor/clinics
US-13	As a doctor, I want to define time slots per clinic so patients can only book real availability.	Must Have	DoctorClinicDetailsPage → /api/doctor/clinics/:id/slots
US-14	As a doctor, I want a dashboard summarizing today's/this week's appointments and my rating so I can plan my day.	Should Have	DoctorDashboard → GET /api/doctor/dashboard
US-15	As an admin, I want to verify pending doctors and clinics before they appear to patients so quality is controlled.	Must Have	AdminDashboard → PUT /api/admin/doctors/:id/verify, /api/admin/clinics/:id/verify
US-16	As an admin, I want to manage user accounts (activate/deactivate) so I can moderate the platform.	Must Have	AdminDashboard → /api/admin/users/:id, /:id/activate
US-17	As an admin, I want to view platform analytics and reports so I understand overall usage.	Should Have	AdminDashboard → GET /api/admin/analytics, /api/admin/reports/*
US-18	As an admin, I want to track and resolve patient complaints so issues don't go unaddressed.	Must Have	AdminDashboard → /api/admin/complaints/*

3.3 Functional Requirements

A. Authentication & Account Requirements

ID	Requirement	Priority	Route(s)
FR-01	The system shall allow a new user to register as a Patient, capturing username, email, password, first/last name, date of birth, phone number, and address.	Must Have	POST /api/auth/register/patient
FR-02	The system shall allow a new user to register as a Doctor, capturing username, email, password, first/last name, specialty, license number, and bio.	Must Have	POST /api/auth/register/doctor
FR-03	The system shall allow a new user to register as an Admin (username, email, password).	Should Have	POST /api/auth/register/admin
FR-04	The system shall reject registration if the email or username already exists.	Must Have	POST /api/auth/register/*
FR-05	The system shall authenticate a user by email and password and return a signed JWT containing the user's id and role.	Must Have	POST /api/auth/login
FR-06	The system shall reject login for a deactivated account, even with correct credentials.	Must Have	POST /api/auth/login
FR-07	The system shall redirect an authenticated user to the dashboard matching their role (Patient / Doctor / Admin) and block access to routes reserved for other roles.	Must Have	App.jsx ProtectedRoute / PublicRoute

B. Patient Requirements

ID	Requirement	Priority	Route(s)
FR-08	The system shall let a patient view and edit their profile (name, DOB, phone, address) and upload a	Must Have	GET/PUT /api/patient/profile, POST /upload-profile-image

ID	Requirement	Priority	Route(s)
	profile image.		
FR-09	The system shall let a patient search doctors filtered by specialty, location (city/country), and minimum rating, with pagination.	Must Have	GET /api/patient/doctors/search
FR-10	The system shall only return verified (isVerified = true) doctors in search results.	Must Have	GET /api/patient/doctors/search
FR-11	The system shall let a patient view a clinic's full details, including doctor info, address, operating hours, and verification status.	Must Have	GET /api/patient/clinics/:clinicId
FR-12	The system shall let a patient view a doctor's available slots for a given clinic and date, excluding already-booked and disabled slots.	Must Have	GET /api/patient/doctors/:doctorId/slots
FR-13	The system shall let a patient create an appointment for a valid clinic + slot combination, storing patient, doctor, clinic, and time details.	Must Have	POST /api/patient/appointments
FR-14	The system shall let a patient view, update, and cancel (soft-delete via status change) their own appointments only.	Must Have	GET/PUT/DELETE /api/patient/appointments/:id
FR-15	The system shall let a patient submit, view, edit, and delete a review (rating + comment) for a doctor.	Should Have	POST/GET/PUT/DELETE /api/patient/reviews
FR-16	The system shall let anyone view a doctor's paginated public reviews.	Should Have	GET /api/patient/doctors/:doctorId/reviews
FR-17	The system shall let a patient create, view, update, and delete their own medical history records (diagnosis, treatment, prescription, notes).	Should Have	/api/patient/medical-history
FR-18	The system shall let a patient view paginated notifications, mark a notification as read, and delete it.	Should Have	/api/patient/notifications

C. Doctor Requirements

ID	Requirement	Priority	Route(s)
FR-19	The system shall let a doctor view and edit their own profile (specialty, license number, bio) and upload a profile image.	Must Have	GET/PUT /api/doctor/profile, POST /upload-profile-image
FR-20	The system shall let a doctor create, view, update, and delete their own clinics; new clinics default to unverified.	Must Have	/api/doctor/clinics
FR-21	The system shall let a doctor define, update, and delete time slots for a specific clinic.	Must Have	/api/doctor/clinics/:clinicId/slots
FR-22	The system shall provide a doctor dashboard summarizing today's/weekly/monthly appointment counts, unique patient count, average rating, and up to 5 upcoming appointments.	Should Have	GET /api/doctor/dashboard
FR-23	The system shall let a doctor view, mark read, and delete their own notifications.	Should Have	/api/doctor/notifications

D. Admin Requirements

ID	Requirement	Priority	Route(s)
FR-24	The system shall let an admin list all users, optionally filtered by role, with pagination.	Must Have	GET /api/admin/users
FR-25	The system shall let an admin view, edit, deactivate, and reactivate a specific user account.	Must Have	GET/PUT /api/admin/users/:id, DELETE, PUT /:id/activate
FR-26	The system shall let an admin list pending (unverified) doctors and clinics and approve them.	Must Have	GET /api/admin/doctors/pending, PUT /:id/verify; equivalent for clinics
FR-27	The system shall let an admin view platform-wide analytics: total users, patients, doctors, verified doctors, clinics, appointments, and pending verification counts.	Should Have	GET /api/admin/analytics
FR-28	The system shall let an admin generate appointment and user reports.	Should Have	GET /api/admin/reports/appointments, /reports/users
FR-29	The system shall let an admin list, view, create, update, and resolve patient complaints, filterable by status.	Must Have	/api/admin/complaints/*

E. Messaging Requirements

ID	Requirement	Priority	Route(s)
FR-30	The system shall let a patient or doctor view the full message history with a specific other user, sorted chronologically.	Must Have	GET /api/chat/history/:otherUserId
FR-31	The system shall let a patient or doctor view a list of their conversations, each showing the other party, last message, and unread count.	Must Have	GET /api/chat/conversations
FR-32	The system shall let a patient or doctor send a message to another user.	Must Have	POST /api/chat/send

3.4 Non-Functional Requirements

ID	Category	Description	Priority
NFR-01	Security	All authenticated endpoints shall require a valid JWT bearer token; requests without one, or with an invalid/expired one, shall be rejected with 401.	Critical
NFR-02	Security	Role-restricted endpoints (Patient/Doctor/Admin) shall verify the role claim in the JWT before processing the request, not just presence of a token.	Critical
NFR-03	Security	Passwords shall not be logged or returned in any API response payload.	High
NFR-04	Security	CORS shall be configured so the frontend origin can call the API in development; a production deployment shall restrict this to known origins.	Medium
NFR-05	Usability	The layout shall be responsive across mobile (<768px), tablet (768–1024px), and desktop (>1024px) breakpoints using Tailwind's responsive utilities.	High
NFR-06	Usability	Every asynchronous action (search, booking, login) shall provide	High

ID	Category	Description	Priority
		visible feedback via a loading state or a toast on success/failure.	
NFR-07	Reliability	Client-side route protection (ProtectedRoute) shall redirect unauthenticated or wrong-role users before any protected content renders.	High
NFR-08	Reliability	The API shall respond with a structured 404 for unknown endpoints and for missing resources (e.g. appointment not found).	Medium
NFR-09	Maintainability	Frontend code shall follow a feature-based structure: pages/, components/{admin,doctor,register,ui}, services/, contexts/, layouts/.	High
NFR-10	Maintainability	All React components shall be functional components using hooks; no class components.	Medium
NFR-11	Performance	The Vite dev server and production build shall support hot module replacement and code-split routing where practical.	Medium
NFR-12	Data Integrity	Because data is stored in memory, the server acknowledges (via documentation) that all data resets on restart; this is an accepted constraint for the current scope, not a defect.	High
NFR-13	Portability	The API base URL shall be configurable via the VITE_API_BASE_URL environment variable rather than hard-coded.	Medium

4. System Analysis & Design

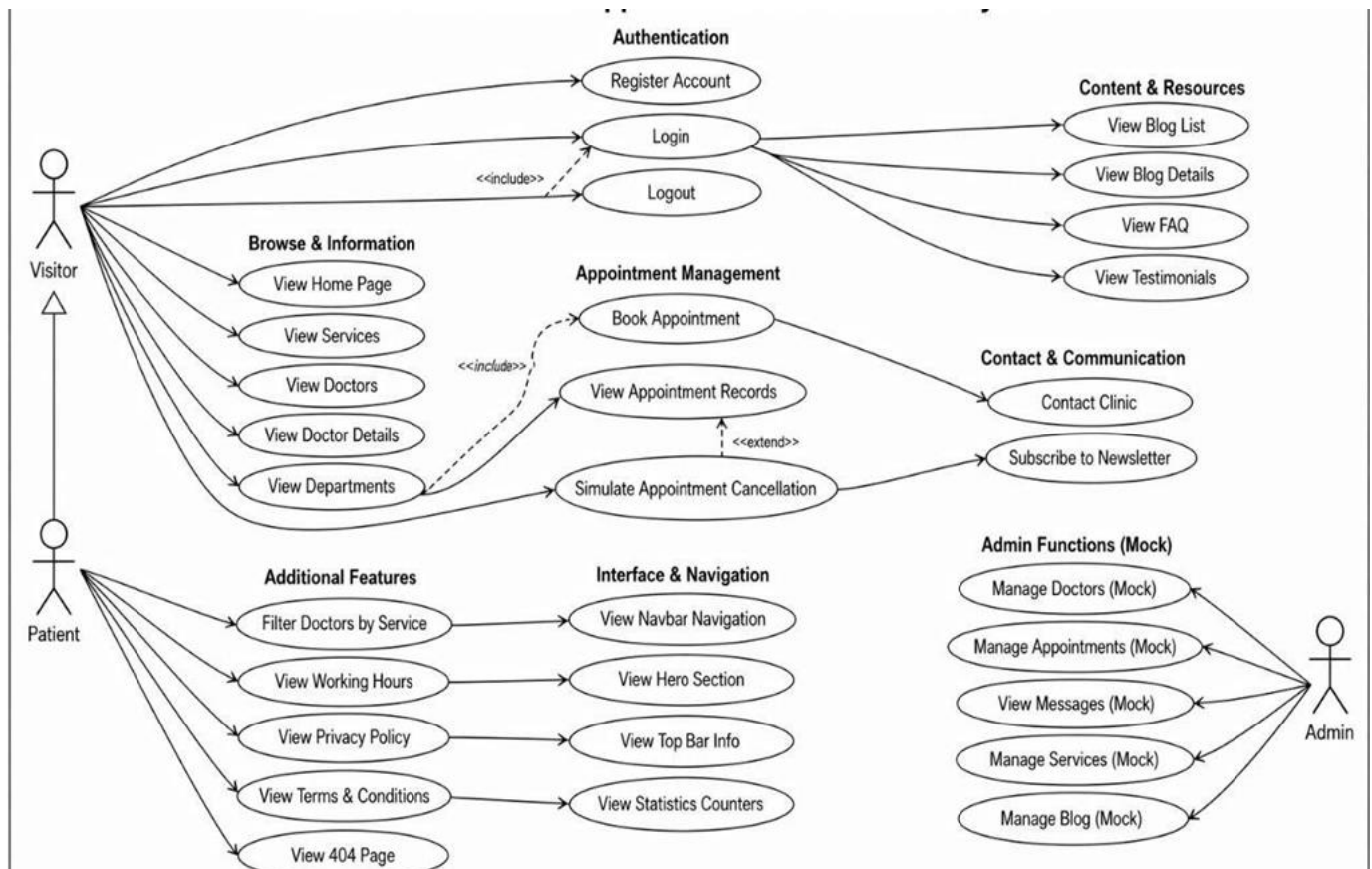
4.1 Problem Statement & Objectives

Problem: Patients typically discover and book medical appointments through phone calls or in-person visits, with no visibility into real-time doctor/clinic availability, and no simple way to message a provider or track their own medical history digitally.

Objective: CareConnect provides a self-service web platform where patients can discover verified doctors and clinics, view real slot-level availability, book and manage appointments, message providers directly, and maintain their own medical history — while doctors manage their clinics and schedules, and admins maintain platform quality through a verification and moderation workflow.

- Implement a complete three-role system (Patient, Doctor, Admin) with JWT-based authentication and role-aware routing.
- Model a realistic clinic/slot booking domain: a doctor can run multiple clinics, each with its own operating hours and bookable time slots.
- Support the full patient lifecycle: search → view profile/slots → book → manage → review.
- Give admins a verification gate so unverified doctors/clinics are never visible to patients.
- Keep the stack approachable for a solo learner: no external database, no payment gateway, no real-time transport layer.

Use Case Diagram



4.2 Use Case Descriptions

[Figure 4.1 — CareConnect Use-Case Diagram — actors: Visitor, Patient, Doctor, Admin — to be produced in a dedicated diagramming tool such as draw.io or Lucidchart]

UC-01 — Book Appointment

Field	Detail
Goal	Allow an authenticated patient to reserve an available time slot at a specific clinic.
Actor	Patient
Route	/booking/:doctorId → BookingFlow (patient role required)
Precondition	Patient is logged in. Doctor is verified. Target clinic has at least one open, non-disabled, non-booked slot for the chosen date.
Postcondition	A new appointment is created with status "Confirmed"; the slot becomes unavailable for that date/clinic combination.
Main Scenario	1. Patient opens a doctor's profile and selects a clinic. 2. Patient picks a date and an open slot. 3. Patient confirms notes (optional). 4. POST /api/patient/appointments. 5. Confirmation toast; appointment appears in the patient dashboard.
Alternate A	Selected clinic or slot is invalid (already booked/removed) → API returns 400; UI shows an error and refreshes available slots.
Alternate B	Patient is not authenticated → ProtectedRoute redirects to /login before the flow starts.

UC-02 — Search & View Doctor Profile

Field	Detail
Goal	Let a visitor or patient find a doctor by specialty/location/rating and view their full profile, clinics, and reviews.
Actor	Visitor or Patient
Route	/doctors → DoctorSearchPage, /doctors/:doctorId → DoctorProfilePage
Precondition	None — search and profile viewing are public.
Postcondition	Doctor list or profile rendered with live data from the API.
Main Scenario	1. User enters filters (specialty, location, rating). 2. GET /api/patient/doctors/search. 3. User clicks a doctor card. 4. GET doctor profile, clinics, and reviews. 5. "Book Appointment" CTA links into UC-01 (login required if not authenticated).
Alternate A	No doctors match filters → empty-state message shown, filters remain editable.

UC-03 — Doctor Verification

Field	Detail
Goal	Ensure only vetted doctors and clinics are visible to patients.
Actor	Admin
Route	/dashboard/admin → AdminDashboard
Precondition	Admin is authenticated. At least one doctor or clinic has isVerified = false.
Postcondition	Doctor/clinic isVerified flag is set to true and, for a first-time doctor verification, a default rating

Field	Detail
	is assigned.
Main Scenario	1. Admin opens the pending-doctors (or pending-clinics) list. 2. GET /api/admin/doctors/pending. 3. Admin approves an entry. 4. PUT /api/admin/doctors/:id/verify. 5. Doctor becomes searchable and bookable.
Alternate A	Admin instead manages a user account (activate/deactivate) via the equivalent users endpoints — same page, different panel.

UC-04 — Doctor–Patient Messaging

Field	Detail
Goal	Allow a patient and a doctor to exchange messages tied to their relationship (not necessarily a specific appointment).
Actor	Patient or Doctor
Route	/chat → ChatPage (Patient or Doctor role required)
Precondition	Both users are registered; at least one prior message or a known contact (e.g. from an appointment) exists to start a conversation.
Postcondition	Message stored server-side; conversation list and unread counts update for both parties.
Main Scenario	1. User opens Chat and sees a conversation list (GET /api/chat/conversations). 2. User selects a conversation and loads history (GET /api/chat/history/:otherUserId). 3. User sends a message (POST /api/chat/send).
Alternate A	No conversations yet → empty state shown; the flow doesn't currently expose a way to start a chat from scratch outside an existing relationship.

4.3 Software Architecture

CareConnect follows a classic decoupled SPA + REST API architecture:

- Frontend: a React 18 SPA (Vite build) using React Router DOM v7 for client-side routing, with a single AuthContext (React Context + useState/useEffect) holding the current user and JWT.
- Backend: a stateless Express REST API. Each domain (auth, patient, doctor, admin, chat) has its own router file; a small authRequired(...roles) middleware validates the JWT and role on protected routes.
- Data: held in plain JavaScript arrays in server/data/store.js (users, patients, doctors, clinics, slots, appointments, reviews, medicalHistory, notifications, messages, complaints) with simple find/filter helper functions — there is no database, so all data resets when the server restarts.
- Communication: the frontend calls the API over HTTP via an Axios instance (services/api.js) with the base URL from VITE_API_BASE_URL; authenticated calls attach the JWT as a Bearer token.

4.4 Route → Component Map

Route	Page Component	Access	Notes
/	HomePage	Public	Hero, specialties, featured doctors, how-it-works, promotions, footer
/doctors	DoctorSearchPage	Public	Filter by specialty/location/rating

Route	Page Component	Access	Notes
/doctors/:doctorId	DoctorProfilePage	Public	Profile, clinics, slots, reviews
/clinics	ClinicSearchPage	Public	Browse clinics
/clinics/:clinicId	ClinicPage	Public	Clinic details, operating hours
/booking/:doctorId	BookingFlow	Patient only	Clinic → date → slot → confirm
/dashboard/patient	PatientDashboard	Patient only	Appointments, reviews, medical history, notifications, profile
/dashboard/doctor	DoctorDashboard	Doctor only	Clinics, slots, appointment summary, notifications
/dashboard/doctor/clinics/:clinicId	DoctorClinicDetailsPage	Doctor only	Manage a single clinic's slots and hours
/dashboard/admin	AdminDashboard	Admin only	Users, pending verifications, analytics, complaints
/chat	ChatPage	Patient or Doctor	Conversation list + message thread
/login	LoginPage	Public (redirects if authenticated)	Role-based post-login redirect
/register	RegisterPage	Public (redirects if authenticated)	Patient / Doctor registration forms
*	NotFound	Public	React Router catch-all

4.5 Data Model

Entity	Key Fields	Relationships
User	id, username, email, password, role (Patient Doctor Admin), isActive, createdAt	Has one Patient or Doctor profile depending on role
Patient	id, userId, firstName, lastName, dateOfBirth, phoneNumber, address, profileImageUrl	Belongs to User; has many Appointments, Reviews, MedicalHistory records
Doctor	id, userId, firstName, lastName, specialty, licenseNumber, bio, profileImageUrl, isVerified, rating	Belongs to User; has many Clinics, Appointments, Reviews
Clinic	id, doctorId, name, address, city, country, phoneNumber, email, isVerified, operatingHours[]	Belongs to Doctor; has many Slots, Appointments
Slot	slotId, clinicId, startTime, endTime, disabled	Belongs to Clinic; referenced by Appointments
Appointment	id, userId, patientId, patientName, doctorId, doctorName, clinicId, clinicName, slotId, appointmentDate, startTime, endTime, status, notes, createdAt	Belongs to Patient, Doctor, Clinic, and Slot
Review	id, patientId, patientName, doctorId, doctorName,	Belongs to Patient and Doctor

Entity	Key Fields	Relationships
	rating, comment, createdAt	
MedicalHistory	id, patientId, patientName, doctorId, doctorName, visitDate, diagnosis, treatment, prescription, notes, createdAt	Belongs to Patient (and referencing Doctor)
Notification	id, userId, title, message, type, isRead, createdAt	Belongs to User
Message	id, senderId, receiverId, message, sentAt, isRead	Between two Users (Patient ↔ Doctor)
Complaint	id, status (Pending Resolved), createdAt, resolvedAt, + submitted fields	Managed by Admin

4.6 API Reference

Auth

Method	Endpoint	Description	Auth
POST	/api/auth/login	Authenticate and receive a JWT + role	No
POST	/api/auth/register/patient	Register a new patient account	No
POST	/api/auth/register/doctor	Register a new (unverified) doctor account	No
POST	/api/auth/register/admin	Register a new admin account	No

Patient

Method	Endpoint	Description	Auth
GET/PUT	/api/patient/profile	View/update own patient profile	Patient
POST	/api/patient/upload-profile-image	Upload a profile image (mock URL)	Patient
GET	/api/patient/doctors/search	Search verified doctors (specialty/location/rating, paginated)	No
GET	/api/patient/doctors/:doctorId/slots	List open slots for a clinic/date	No
GET/POST	/api/patient/appointments	List own appointments / create a new one	Patient
GET/PUT/DELETE	/api/patient/appointments/:id	View/update/cancel own appointment	Patient
GET/POST	/api/patient/medical-history	List / add own medical history records	Patient
GET/PUT/DELETE	/api/patient/medical-history/:id	View/update/delete a record	Patient
GET/POST	/api/patient/reviews	List own reviews / submit a review	Patient
PUT/DELETE	/api/patient/reviews/:id	Update/delete own review	Patient
GET	/api/patient/doctors/:doctorId/reviews	Public paginated reviews for a doctor	No
GET	/api/patient/clinics/:clinicId	Public clinic details	No
GET	/api/patient/notifications	Paginated own notifications	Patient
PUT/DELETE	/api/patient/notifications/:id	Mark read / delete a notification	Patient

Doctor

Method	Endpoint	Description	Auth
GET/PUT	/api/doctor/profile	View/update own doctor profile	Doctor
POST	/api/doctor/upload-profile-image	Upload a profile image (mock URL)	Doctor
GET/POST	/api/doctor/clinics	List / create own clinics	Doctor
GET/PUT/DELETE	/api/doctor/clinics/:id	View/update/delete own clinic	Doctor
GET/POST	/api/doctor/clinics/:clinicId/slots	List / create slots for a clinic	Doctor
PUT/DELETE	/api/doctor/clinics/:clinicId/slots/:slotId	Update/delete a slot	Doctor
GET	/api/doctor/dashboard	Appointment/patient/rating summary	Doctor
GET	/api/doctor/notifications	Paginated own notifications	Doctor
PUT/DELETE	/api/doctor/notifications/:id	Mark read / delete a notification	Doctor

Admin

Method	Endpoint	Description	Auth
GET	/api/admin/users	Paginated user list, filterable by role	Admin
GET/PUT/DELETE	/api/admin/users/:id	View/update/deactivate a user	Admin
PUT	/api/admin/users/:id/activate	Reactivate a deactivated user	Admin
GET	/api/admin/doctors/pending	List unverified doctors	Admin
PUT	/api/admin/doctors/:id/verify	Verify a doctor	Admin
GET	/api/admin/clinics/pending	List unverified clinics	Admin
PUT	/api/admin/clinics/:id/verify	Verify a clinic	Admin
GET	/api/admin/analytics	Platform-wide summary counts	Admin
GET	/api/admin/reports/appointments	Full appointment report	Admin
GET	/api/admin/reports/users	Full user report	Admin
GET/POST	/api/admin/complaints	List (filterable) / file a complaint	Admin
GET/PUT/DELETE	/api/admin/complaints/:id	View/update/delete a complaint	Admin
PUT	/api/admin/complaints/:id/resolve	Mark a complaint resolved	Admin

Chat

Method	Endpoint	Description	Auth
GET	/api/chat/history/:otherUserId	Full message history with one other user	Patient/Doctor
GET	/api/chat/conversations	List of conversations with last message + unread count	Patient/Doctor
POST	/api/chat/send	Send a message to another user	Patient/Doctor

5. Implementation

5.1 Repository Structure

Path	Purpose
frontend/src/pages/	One file per route: HomePage, DoctorSearchPage, DoctorProfilePage, ClinicSearchPage, ClinicPage, BookingFlow, PatientDashboard, DoctorDashboard, DoctorClinicDetailsPage, AdminDashboard, ChatPage, LoginPage, RegisterPage, NotFound
frontend/src/components/	Shared top-level components: Header, Navbar, Footer, HeroSection, HowItWorks, FeaturedDoctors, SpecialtiesSection, PromotionsSection
frontend/src/components/admin/	Admin dashboard sections (users, verifications, analytics, complaints)
frontend/src/components/doctor/	Doctor clinic-management sections
frontend/src/components/register/	Registration form sections (patient vs. doctor forms)
frontend/src/components/ui/	shadcn/Radix UI primitives (template — not written from scratch)
frontend/src/contexts/AuthContext.jsx	Single global auth context: current user, token, login/logout/register helpers
frontend/src/layouts/	MainLayout (public/app shell) and AuthLayout (login/register shell)
frontend/src/services/	Axios-based API modules: api.js (instance + interceptors), auth.js, patient.js, doctor.js, admin.js, chat.js, upload.js, validation.js, types.js, imageUrl.js
frontend/src/App.jsx	Route table, ProtectedRoute / PublicRoute guards
server/server.js	Express app entry point; mounts auth/patient/doctor/admin/chat routers
server/middleware/auth.js	JWT creation (createToken) and verification middleware (authRequired)
server/routes/	One router per domain: auth.js, patient.js, doctor.js, admin.js, chat.js
server/data/store.js	In-memory arrays for every entity, ID counters, and lookup/helper functions

5.2 Coding Notes

- All components are functional, using hooks (useState, useEffect, custom context hooks) — no class components.
- Routing distinguishes public routes, role-protected routes (ProtectedRoute with allowedRoles), and auth-only routes that redirect an already-logged-in user away from /login and /register (PublicRoute).
- The Express layer uses a small, repeated authRequired(...roles) middleware pattern per router rather than a single global auth gate, keeping each route's access rule explicit and readable.
- IDs are generated in-memory via a simple nextId(counter) switch in store.js rather than a database auto-increment or UUID scheme — acceptable for a mock API, but not suitable to carry into a real persistence layer as-is.
- Passwords are compared with plain string equality (user.password !== password) in the current implementation. This is explicitly flagged as an AI-assisted shortcut appropriate only for a local learning/demo environment (see Section 6.3, Known Limitations).

5.3 Environment & Execution

Item	Detail
------	--------

Item	Detail
Install	npm install && npm run install:all (installs root, frontend, and server dependencies)
Run both (dev)	npm run dev — starts server (blue) and frontend (green) concurrently
Run separately	npm run dev:server (port 3001) / npm run dev:frontend (port 5173)
Frontend URL	http://localhost:5173
API URL	http://localhost:3001
Env config	frontend/.env — VITE_API_BASE_URL=http://localhost:3001
Production build	npm run build (frontend) → static output for deployment
Lint	npm run lint (ESLint, frontend)

5.4 Demo Accounts

Role	Email	Password	Notes
Patient	john@example.com	password123	Has one existing confirmed appointment and a review on record
Doctor (verified)	sarah@medicare.com	password123	Cardiologist at City Heart Center — good path for testing booking
Doctor (pending)	pending@medicare.com	password123	Dermatologist, isVerified = false — use to test the admin verification flow
Admin	admin@careconnect.com	admin123	Full access to users, verifications, analytics, and complaints

6. Testing & Quality Assurance

6.1 Manual Test Plan

No automated test suite exists yet (see Section 7.3). The following manual test cases cover the core flows and map to the functional requirements in Section 3.3.

TC ID	Req. ID	Test Case	Expected Result
TC-01	FR-05	Log in with sarah@medicare.com / password123	JWT returned; redirected to /dashboard/doctor
TC-02	FR-06	Attempt login on a deactivated account	401 response: "Account is deactivated."
TC-03	FR-04	Register a patient with an email already in use	400 response: "Email already exists."
TC-04	FR-09/FR-10	Search doctors by specialty=Dermatology	James Wilson (unverified) is excluded from results
TC-05	FR-12	View slots for a clinic/date where one slot is already booked	Booked slot excluded from the returned list
TC-06	FR-13	Book an appointment with a valid clinicId/slotId	201-style success; appointment status = Confirmed
TC-07	FR-13	Book with a slotId that doesn't belong to the given clinicId	400 response: "Invalid clinic or slot."
TC-08	FR-14	Patient A attempts to cancel Patient B's appointment ID	404 — appointment not found for this user
TC-09	FR-26	Admin verifies pending doctor (pending@medicare.com)	isVerified → true; doctor now appears in search
TC-10	FR-07	Log in as Patient, manually navigate to /dashboard/admin	Redirected to / (patient not authorized for admin route)
TC-11	NFR-01	Call a protected endpoint with no Authorization header	401 Unauthorized
TC-12	NFR-05	Load the homepage at a 375px viewport width	No horizontal scroll; nav collapses appropriately
TC-13	FR-30/31/32	Send a message, then load conversations for both sender and receiver	Message appears in both users' history; unread count increments for receiver

6.2 Known Limitations

- Data does not persist — restarting the server resets all users, appointments, messages, etc. to the seed data in store.js.
- Passwords are stored and compared in plaintext; this must be replaced with hashing (e.g. bcrypt) before any real deployment.
- No automated unit, integration, or end-to-end tests exist yet.
- No rate limiting, input sanitization, or centralized error-handling middleware on the API.
- Uploaded profile images are mocked (a fixed Unsplash URL is returned) rather than actually processing a file upload.

7. Final Presentation & Reports

7.1 User Manual (Patient Journey)

- Open the CareConnect homepage and browse featured doctors and specialties, or go to "Find a Doctor".
- Use the specialty, location, and rating filters on the Doctor Search page to narrow results.
- Click a doctor card to view their full profile: bio, specialty, clinics, and patient reviews.
- Click "Book Appointment" — if not logged in, you'll be sent to Login/Register first.
- In the booking flow, choose a clinic, pick a date, choose an open time slot, and confirm.
- From your Patient Dashboard, view upcoming/past appointments, cancel if needed, leave a review, and manage your medical history.
- Use the Chat page to message your doctor directly.

7.2 User Manual (Doctor & Admin Journeys)

Doctor: after logging in, a doctor lands on their Dashboard, where they can register one or more clinics, define operating hours and time slots per clinic, and monitor an at-a-glance summary of today's/this month's appointments, unique patient count, and average rating. Doctors also use the Chat page to respond to patient messages.

Admin: after logging in, an admin lands on the Admin Dashboard, where pending doctors and clinics can be reviewed and verified, user accounts can be activated/deactivated, platform analytics can be reviewed, and patient complaints can be tracked through to resolution.

7.3 Future Enhancements

- Replace the in-memory store with a real database (e.g. PostgreSQL or MongoDB) for durable persistence.
- Hash passwords with bcrypt and add refresh tokens / shorter-lived access tokens.
- Add an automated test suite: component tests (React Testing Library) and API tests (e.g. Supertest) for the Express routes.
- Real file uploads for profile images instead of a mocked static URL.
- Real-time chat delivery (WebSockets) instead of polling-style REST calls.
- Email/SMS notifications for booking confirmations and cancellations.
- Payment integration for paid consultations.

7.4 Technical Summary

Architecture: React 18 SPA (Vite) with React Router DOM v7, a single AuthContext for session state, Tailwind CSS + Radix/shadcn UI, and Axios for API calls, talking to a stateless Express + JWT REST API backed by an in-memory data store.

Deployment target: any static host for the built frontend (e.g. Vercel/Netlify) plus any Node-capable host for the Express API (e.g. Render/Railway), with VITE_API_BASE_URL pointed at the deployed API origin.

Appendix A: Glossary

Term	Definition
SPA	Single Page Application — loads once and updates the view client-side without full page reloads.
JWT	JSON Web Token — a signed token used here to carry a user's id and role for stateless authentication.
Slot	A discrete bookable time window at a specific clinic (e.g. 09:00–09:30).
Clinic	A physical location tied to exactly one doctor, with its own address, operating hours, and slots.
Verification	The admin action of marking a doctor or clinic as trustworthy/visible to patients (isVerified = true).
ProtectedRoute	A frontend routing guard that redirects unauthenticated or wrong-role users away from a page.
Mock API	An API that simulates real backend behavior using in-memory data instead of a persistent database.
shadcn/ui	A library of copy-in React UI components built on Radix UI primitives, used here as a styling template.

Appendix B: Demo Accounts (Quick Reference)

Role	Email	Password
Patient	john@example.com	password123
Doctor	sarah@medicare.com	password123
Admin	admin@careconnect.com	admin123

Appendix C: References

- IEEE Std 830-1998: IEEE Recommended Practice for Software Requirements Specifications.
- React documentation — <https://react.dev>
- React Router documentation — <https://reactrouter.com>
- Express documentation — <https://expressjs.com>
- Tailwind CSS documentation — <https://tailwindcss.com>
- shadcn/ui — <https://ui.shadcn.com>
- Radix UI — <https://www.radix-ui.com>